

Adaptive BLE Connection Management: Balancing Energy Consumption and Quality of Service

Milica Puhalo

31 August 2026

1 Introduction

The goal of this project is to implement an adaptive Bluetooth Low Energy (BLE) mechanism that dynamically balances two objectives: minimizing energy consumption while maintaining a Quality of Service (QoS) level.

The implemented solution is based on the research paper *AdaptaBLE: Adaptive control of data rate, transmission power, and connection interval in Bluetooth Low Energy*[1]. The AdaptaBLE algorithm jointly adjusts three radio parameters:

- **Connection interval** (T_{CI}) — how often the master and slave exchange packets;
- **Physical-layer data rate** (PHY) — 2 M, 1 M, 500 k or 125 k (coded);
- **Transmission power** — the output power used for transmissions.

By estimating the effective number of connection events a packet spends waiting in the controller (the Estimated Connection Interval, ECI_i), the algorithm can determine whether the link is meeting the latency target. When the link quality is good, the algorithm reduces energy by increasing the connection interval, lowering the data rate or reducing TX power. When the link quality degrades, it improves reliability by shortening the connection interval, raising the data rate or increasing TX power.

This document describes the main software mechanisms that were implemented on the ESP32-S3 platform using the NimBLE host stack.

2 Overview of project components

The central device uses the Generic Access Profile (GAP) layer to discover, connect to and disconnect from a BLE peripheral. The implementation follows the event-driven model: the application registers callback functions with the stack and the stack invokes them when GAP events occur.

2.1 GAP State Machine

A small state machine tracks the high-level connection status:

State	Meaning
GAP_STATE_IDLE	No active scan or connection
GAP_STATE_SCANNING	Actively scanning for peripherals
GAP_STATE_CONNECTING	Connection request in progress
GAP_STATE_CONNECTED	Connection established

Table 1: GAP states used by the central device.

Transitions between states are triggered by events such as `BLE_GAP_EVENT_DISC`, `BLE_GAP_EVENT_CONNECT` and `BLE_GAP_EVENT_DISCONNECT`. The event handler is registered with `ble_gap_event_handler()` and receives a `struct ble_gap_event` pointer.

2.2 Library Functions and Callbacks

Connection management is implemented by calling NimBLE GAP functions and passing them callback pointers. The most important calls are:

- `ble_gap_disc(..., scan_event_handler, ...)` starts scanning and reports discovered devices through `scan_event_handler`.
- `ble_gap_connect(..., gap_event_handler, ...)` initiates a connection and reports connection status through `gap_event_handler`.
- `ble_gap_update_params(...)` requests a new connection interval, latency and supervision timeout.
- `ble_gap_set_prefered_le_phy(...)` requests a PHY update for the data rate.

The event handler is responsible for storing the connection handle, updating the state variable, notifying the AdaptBLE module of the initial connection interval, and starting GATT discovery once the connection is established.

3 Console Interface

The console allows a user to control the central device at runtime without recompiling or rebooting. The implemented commands are:

Command	Description
<code>scan</code>	Start scanning for BLE peripherals.
<code>stop_scan</code>	Stop an active scan.
<code>connect <address></code>	Connect to a peripheral by Bluetooth address.
<code>disconnect <address name></code>	Disconnect from a connected peripheral.
<code>get_state</code>	Print the current GAP state and, if connected, the peer address and name.

Table 2: Console commands implemented on the central device.

When `disconnect` is issued, the command handler invokes the GAP disconnect function, which eventually triggers the disconnect event. The disconnect event handler then stops the AdaptBLE adaptation task and the dummy-stream GATT task to ensure no further processing or traffic occurs while the device is disconnected.

4 AdaptBLE Algorithm

The core of the project is the AdaptBLE algorithm. It is compiled conditionally under the `ADAPT_BLE` preprocessor flag and runs as a FreeRTOS task that wakes up once per second.

4.1 Latency Estimation

The latency estimator measures, in T_{CI} , units how much time is needed to send a packet from the host to the connected device. Two timestamps are captured:

- T_{ADD} — the moment the host submits an ACL data packet to the controller. It is captured in `ble_hci_trans_hs_acl_tx()` inside `esp_nimble_hci.c`.
- T_{FREE} — the moment the controller reports that the transmission buffer has been freed. It is captured when the `HCI_Number_Of_Completed_Packets` event is received, inside `ble_hs_hci_evt_num_completed_pkts()` in `ble_hs_hci_evt.c`.

These timestamps are pushed into a thread-safe FIFO in `adapt_ble.c`. The AdaptBLE task consumes completed (T_{ADD}, T_{FREE}) pairs and computes:

$$t_{TX} = T_{FREE} - T_{ADD} \quad (1)$$

$$ECI_i = \left\lceil \frac{t_{TX} - t_{CE}}{T_{CI}} \right\rceil \quad (2)$$

where t_{CE} is the maximum connection-event duration and T_{CI} is the current connection interval. The figure below illustrates these values, it is taken from the research paper *Improving the Timeliness of Bluetooth Low Energy in Noisy RF Environments*[2]

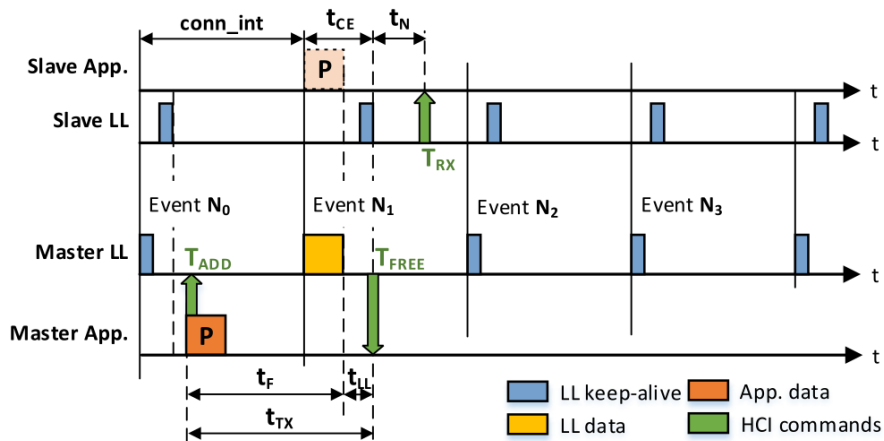


Figure 1: HCI-based latency estimation

4.2 Connection Interval Calculation

Every adaptation round, the algorithm keeps a history of the last M ECI values and selects the worst case:

$$ECI_{max} = \max(ECI_{i-1}, \dots, ECI_{i-M}) \quad (3)$$

The recommended connection interval is then:

$$T_{CI} = \left\lfloor \frac{t_{th} - t_{CE}}{ECI_{max}} \right\rfloor \quad (4)$$

where t_{th} is the latency threshold. The central sends the new interval to the slave using `ble_gap_update_params()`.

4.3 Data Rate and TX-Power Management

Every M rounds the algorithm evaluates the retransmission ratio RT_{ratio} , i.e. the fraction of recent rounds where $ECI_i > 1$. Based on this ratio:

- If $RT_{ratio} > RT_{th,hi}$, the link quality is poor and the algorithm increases energy usage by moving to a higher data rate or higher TX power.
- If $RT_{ratio} < RT_{th,lo}$, the link quality is good and the algorithm reduces energy consumption by moving to a lower data rate or lower TX power.

The algorithm uses a two-dimensional Look-Up Table (LUT). Rows correspond to data rates (2 M, 1 M, 500 k, 125 k) and columns correspond to TX-power levels. The upper-left corner is the lowest-energy operating point and the lower-right corner is the highest-energy, best-link-quality point.

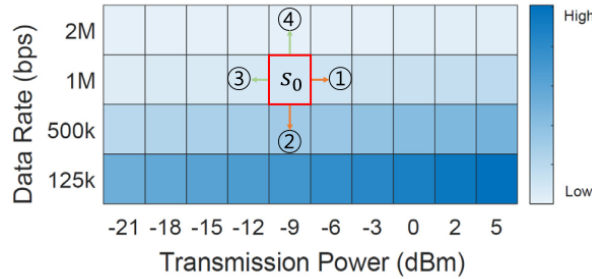


Figure 2: Data rate and transmission power look-up table

PHY changes are sent with `ble_gap_set_preferred_le_phy()`. Because there is no standard BLE control PDU for TX-power updates, the central writes the requested TX-power value to a custom GATT characteristic on the slave. The slave applies the value locally and the central applies the same value locally only after the write is acknowledged.

5 GATT Services

Two custom GATT services are used for the AdaptBLE mechanism.

5.1 TX-Power Service

The peripheral exposes a writable characteristic under a custom 16-bit service UUID 0xAD80 and characteristic UUID 0xAD81. When the central wants to change the slave's TX power, it writes a signed 8-bit dBm value to this characteristic. The peripheral's GATT access callback applies the value with `esp_ble_tx_power_set()` and returns an ATT response. The central uses a write-with-response GATT operation and applies the same TX power locally only inside the acknowledgement callback.

To avoid hard-coding attribute handles, the central discovers the service and characteristic at runtime using `ble_gattc_disc_svc_by_uuid()` followed by `ble_gattc_disc_all_chrs()`. The discovered handle is cached until disconnect.

5.2 Dummy Stream Service

Because the AdaptBLE algorithm needs ACL traffic to measure latency, a second writable characteristic (service UUID 0xAD90, characteristic UUID 0xAD91) is added to the peripheral. The central runs a background task that periodically writes 100-byte dummy packets to this characteristic using Write Without Response. The write operation emulates a continuous data stream (e.g. audio) and provides the ACL packets whose timestamps are captured by the latency estimator.

The stream task only runs while a connection is active; it is stopped on disconnect.

6 Summary

The project implements a complete adaptive BLE link-management system. It combines NimBLE's event-driven GAP/GATT APIs, a runtime console, a latency estimator that hooks into the HCI data path, and a LUT-based adaptation policy for connection interval, data rate and TX power. The design is framed under the `ADAPT_BLE` compilation flag, allowing the adaptive behaviour to be enabled or disabled independently of the basic central/peripheral connection functionality.